

Figure 1: Qt Creator

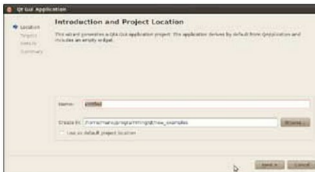


Figure 2: Location selection



Figure 3: Project target selection

```
{
    static int count;
    count++;
    ui->label->setText(QString::number(count));
}
```

This code uses a static variable *count*, incrementing it each time this function is called, and converting it into a *QString*, before setting this as the label widget's text. Press *Ctrl+R* to build and run the application (as in Figure 11). Test it—click the 'Count' button and check the results. Click



Figure 4: Creating class



Figure 5: Code editor



Figure 6: UI editor

'Quit' to end the run.

Custom signals

So we have seen how to use the Qt signal/slot mechanism. You might well ask if we can have our own signals? Yes, we can. Let's try to understand this with a piece of non-GUI sample code. (We described how to create a non-GUI application in the previous article.) Right-click the new project folder in the project window, select *Add New*. Then select the C++ and C++ class from the lists in the pop-up window (Figure 12). Click *Choose*. You will be asked to enter the new class details (Figure 13). Name the class, and use *QObject* as its base class (this is necessary in order to use the signal/slot mechanism in a non-GUI class). Proceed to finish the class creation. The files added to the project are *test.cpp* and *test.h*. First have a look at the final code, and then we will dissect it.

This is *main.cpp*: